

## PROJECT REPORT

# School Administration Management System

A mobile-first platform for daily attendance, academic records, leave, communication and school operations — built offline-first for classroom conditions.

---

**Document type:** System Overview, Requirements & Design Report

**Platform:** React Native (Expo) client · Supabase (PostgreSQL) backend

**Prepared:** September 2026

**Status:** Active development — core modules implemented and functional

# Table of Contents

---

|    |                                       |
|----|---------------------------------------|
| 1  | Introduction                          |
| 2  | Product Overview                      |
| 3  | System Architecture                   |
| 4  | Data Model                            |
| 5  | Roles & Permission Model              |
| 6  | Functional Requirements & Features    |
| 7  | Non-Functional Requirements           |
| 8  | Automation & Scheduled Jobs           |
| 9  | Offline Synchronisation Design        |
| 10 | Security Model                        |
| 11 | User Interface Design — Wireframes    |
| 12 | Navigation Map                        |
| 13 | Quality Assurance & Testing           |
| 14 | Project Status & Build Progress       |
| 15 | Known Limitations & Future Work       |
| 16 | Conclusion                            |
| A  | Appendix A — Permission Key Catalogue |
| B  | Appendix B — Edge Functions           |

# Introduction

---

## 1.1 Purpose of this Document

This report presents the School Administration Management System in full: the problem it solves, the architecture and data model that support it, the complete catalogue of functional and non-functional requirements, the roles and permissions that govern access, and the screens through which staff interact with it. It is written to be handed to an audience unfamiliar with the codebase — reviewers, evaluators, or school stakeholders — as a single, self-contained account of what has been built and why.

## 1.2 Project Background

Primary schools of this scale (roughly 900 students across 30 classes and 35 staff) traditionally rely on paper attendance registers, handwritten mark books, and word-of-mouth or notice-board communication. These methods are slow to compile, error-prone to aggregate, and give school leadership no real-time visibility into attendance risk, staff coverage gaps, or outstanding administrative work. This system replaces that paper trail with a single mobile application usable by every staff role in the school, from a class teacher marking a register to the principal reviewing whole-school attendance trends.

## 1.3 Scope

The system in its current phase is a **staff-facing administrative application**. It covers attendance (student and staff), academic structure, marks and examinations, student promotion, leave management, staff records and accounts, announcements, events and the school diary, inventory, analytics and reporting, notifications, and a customisable home dashboard. A parent-facing companion application is recognised as a later phase; the guardian data model is already designed to support it without structural change, but no parent-facing screens are in scope for this phase.

## 1.4 Intended Audience

This document serves project evaluators and reviewers, school leadership considering adoption, and any developer joining the project who needs a system-level orientation before reading source code.

## Product Overview

### 2.1 Problem Statement

A class teacher must record attendance for roughly 35 students every morning, often in a classroom with unreliable or no network connectivity. School leadership needs to know, within minutes, which classes have not yet marked attendance, which students are at risk of chronic absence, which staff are on leave and whether their classes are covered, and what needs a decision today. None of this is possible from a paper register until someone manually compiles it — usually too late to act on.

### 2.2 Solution Summary

The application puts attendance marking, student and staff records, leave approval, marks entry, announcements, events, inventory and analytics into one role-aware mobile app. Every screen a staff member sees is filtered by what their role and scope actually permit — a class teacher sees their own class, a sectional head their grade, the principal the whole school — and that filtering is enforced by the database itself, not just hidden in the interface. The single most important flow, marking a class register in under thirty seconds even with no signal, is treated as the non-negotiable core the rest of the system is built around.

### 2.3 Key Design Principles

| Principle                                             | Reasoning                                                                                                                                               |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Attendance marking works fully offline</b>         | Classroom connectivity is unreliable; a lost marking session directly undermines adoption of the whole system.                                          |
| <b>A class roster defaults to all-present</b>         | Marking 35 students individually, one by one, is too slow for a daily task with a tight window.                                                         |
| <b>Permissions are scoped data, not code branches</b> | Roles and their coverage (school / grade / class / self) change as the school's structure changes; hard-coding them would require a release every time. |
| <b>Nothing scheduled runs on a non-school day</b>     | A public holiday must never generate hundreds of false "absent" records or reminder notifications.                                                      |
| <b>Every write of consequence is audited</b>          | Attendance, marks, leave decisions and account changes are records the school may need to justify later.                                                |

### 2.4 Target Users

| Role            | Typical use of the system                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Principal       | Whole-school visibility: unmarked classes, at-risk students, leave approvals, announcements, accounts, audit log. |
| Vice Principal  | Near-full school-wide access, mirroring the principal's view for day-to-day operations.                           |
| Sectional Head  | Oversight of one grade: marking status, student records, leave visibility, section announcements.                 |
| Class Teacher   | Daily attendance marking, student profiles for their class, marks entry, leave requests.                          |
| Administrator   | Student and staff records, inventory, events, diary, account provisioning.                                        |
| Staff (general) | Directory access, own leave requests, announcements, and events relevant to them.                                 |

# System Architecture

## 3.1 Architecture Overview

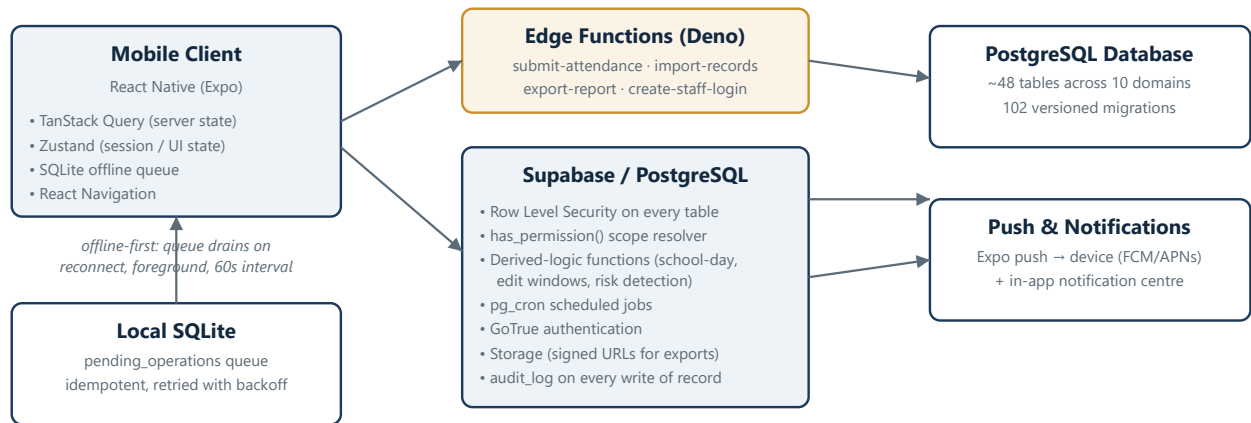


Figure 3.1 — High-level system architecture: offline-capable mobile client, Supabase backend with row-level security enforced at the database, and push notification delivery.

## 3.2 Technology Stack

| Layer              | Choice                                                                                              |
|--------------------|-----------------------------------------------------------------------------------------------------|
| Mobile client      | React Native, Expo managed workflow, TypeScript (strict mode)                                       |
| Backend platform   | Supabase — PostgreSQL, GoTrue authentication, Row Level Security, Storage, Edge Functions, pg_cron  |
| Offline store      | SQLite via expo-sqlite, wrapped in a custom idempotent sync queue                                   |
| Server state       | TanStack Query                                                                                      |
| Local UI state     | Zustand                                                                                             |
| Navigation         | React Navigation (bottom tabs + native stack)                                                       |
| Forms & validation | React Hook Form with Zod schemas, shared between client and Edge Functions                          |
| Push notifications | Expo Notifications over FCM (Android) and APNs (iOS)                                                |
| Dates & timezone   | date-fns / date-fns-tz; all timestamps stored as timestamptz, school timezone held in configuration |
| PDF export         | expo-print (HTML → PDF) with the native OS share sheet                                              |
| Testing            | Vitest (logic), React Native Testing Library (components), pgTAP (database policy tests)            |
| Migrations         | Supabase CLI migrations, versioned in-repo (102 files to date)                                      |

## 3.3 Rules of Construction

- **No business logic in the client.** Permission checks, school-day derivation, attendance locking and risk detection all live in PostgreSQL functions or Edge Functions — a client can be rebuilt or bypassed and the rules still hold.

- **No unscoped reads.** Queries select explicit columns so that what a role can see stays a deliberate, auditable decision, not an accident of a wildcard query.
- **Every table has Row Level Security enabled.** A table with no policy is unreadable — the safe default — rather than open by omission.
- **All thresholds and deadlines are data, not code.** Attendance edit windows, risk thresholds, leave entitlements and due times live in configuration tables, editable by the principal, never hard-coded.

## SECTION 4

# Data Model

The schema spans roughly 48 tables across ten domains. All primary keys are UUIDs; records of consequence are soft-deleted via a status column rather than removed outright; every write of consequence is captured in a central audit log.

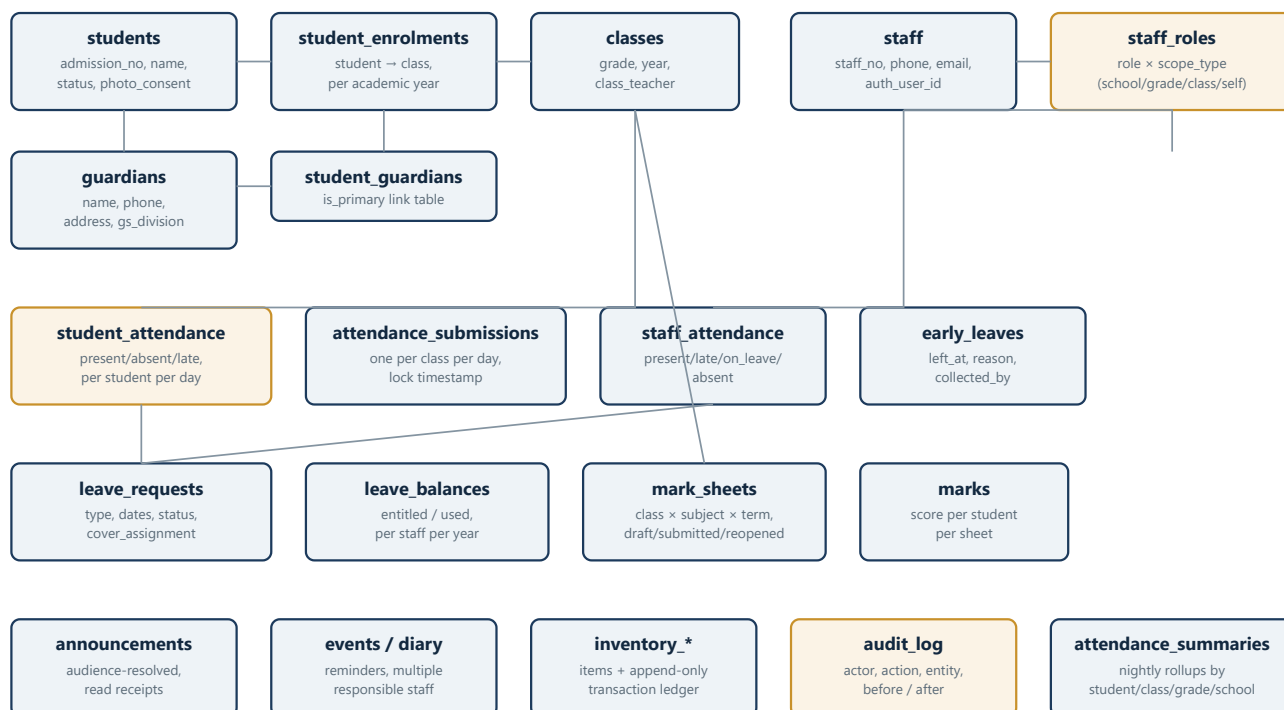


Figure 4.1 — Simplified entity relationship overview (core entities only; gold-outlined entities are cross-cutting: role/scope, submission state, and audit).

## 4.2 Table Catalogue by Domain

| Domain                   | Tables                                                                                                                                                         |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Calendar & configuration | academic_years, terms, calendar_days, school_settings                                                                                                          |
| Academic structure       | grades, classes, subjects, grade_subjects, class_subject_teachers                                                                                              |
| People                   | students, student_enrolments, guardians, student_guardians, staff                                                                                              |
| Roles & scope            | roles, permissions, role_permissions, staff_roles, cover_assignments                                                                                           |
| Attendance               | attendance_submissions, student_attendance, early_leaves, staff_attendance, student_attendance_reopens, staff_attendance_reopens, staff_attendance_submissions |
| Leave                    | leave_types, leave_balances, leave_requests (with maternity-phase chaining)                                                                                    |
| Marks & examinations     | mark_sheets, marks                                                                                                                                             |
| Student record extras    | achievements, memberships, benefits                                                                                                                            |
| Staff, comms & assets    | responsibilities, announcements, announcement_reads, inventory_items, inventory_transactions, events, event_responsible_staff, diary_entries, call_logs        |
| System                   | devices, notifications, audit_log, attendance_summaries, staff_dashboard_prefs                                                                                 |

**Design conventions applied throughout:** UUID primary keys, `created_at` / `updated_at` timestamps maintained automatically, soft-deletion via a `status` column for records with history (students, staff, guardians), and derived values — such as current inventory quantity — are always computed from their transaction ledger rather than stored and risking drift.

# Roles & Permission Model

## 5.1 Model Summary

Access is governed by a scoped role-based permission system rather than fixed code paths. A staff member holds one or more **role grants**, each carrying a **scope** of `school`, `grade`, `class` or `self`. A user may act on a record if any of their active role grants includes the required permission at a scope that covers that record. This is resolved by a single database function, `has_permission()`, called from every Row Level Security policy in the schema — so the rule is enforced identically no matter which client, query, or future interface reaches the data.

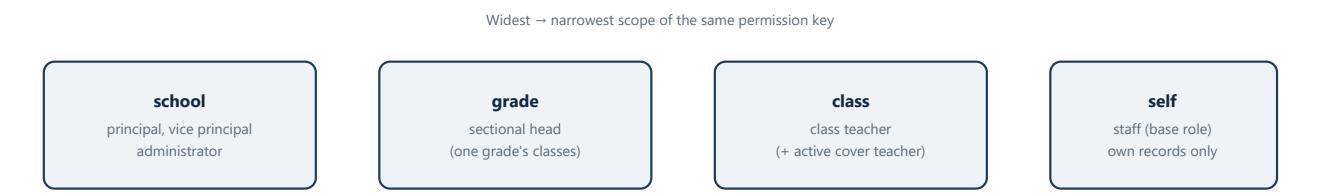


Figure 5.1 — Scope hierarchy: the same permission key (e.g. attendance.view\_board) means something different depending on the scope it was granted at.

## 5.2 Role Catalogue

| Role           | Default scope | Representative permissions                                                                                                                                                                |
|----------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Principal      | School        | All permission keys — the only role that can manage the academic calendar, approve leave, and view the audit log.                                                                         |
| Vice Principal | School        | All keys except leave approval, audit log access, and staff-attendance reopening.                                                                                                         |
| Sectional Head | Grade         | Attendance oversight, full student records, marks review, staff directory, leave visibility and requests, cover assignment, section announcements, analytics — all scoped to their grade. |
| Class Teacher  | Class         | Mark attendance, view/edit their class's students, enter marks, request leave.                                                                                                            |
| Administrator  | School        | Student and staff record management, inventory, events, diary, account provisioning, locked-attendance amendment.                                                                         |
| Staff (base)   | Self          | Staff directory access and leave requests — the floor every account has.                                                                                                                  |

A staff member may hold several role grants at once — e.g. a class teacher who is also a sectional head for their grade. An active cover assignment temporarily grants class-teacher rights over the covered class for its duration, resolved by the same `has_permission()` function.

## 5.3 Permission Keys

Permissions are seeded as data (30 keys across 15 functional areas) rather than encoded in application logic, so adding a permission is a migration and a seed row, not a code change. The full catalogue appears in Appendix A.

## 5.4 Column-Level Restriction

Two categories of information need finer control than row-level filtering alone provides:

- Guardian contact numbers** are exposed only through a dedicated view gated by a specific `student.view_guardian_contact` permission — an ordinary student query never sees a guardian's phone number.

- **Student benefits** (textbooks, uniform vouchers, meal schemes) sit behind their own `student.view_benefits` permission, independent of general student access, since benefit status is considered more sensitive than a name and class.

## Functional Requirements & Features

Each module below states what a user can do, the access control that governs it, and its current implementation status. **NEW** marks capability areas added after the original build specification was written.

### 6.1 Attendance Management **FR-ATT**

- A class teacher marks their roster with every student defaulted to *present*, so only exceptions need to be touched, and submits in one action — designed to complete in under thirty seconds.
- Submission works fully offline: the record is queued locally, confirmed to the user immediately, and synced exactly once when connectivity returns, even if sync is triggered repeatedly.
- Today's submission remains editable for a configurable window (default 60 minutes) after submission; after that, only a principal or administrator can amend it, and must supply a reason.
- A past date is locked outright and can only be reopened by the principal — a reopen is one-shot and re-locks itself the moment a full resubmission succeeds.
- The attendance board lists every class in scope with its marking status, unmarked classes sorted first, with a one-tap reminder to the class teacher.
- Consecutive-absence and term-percentage risk detection runs daily and notifies the class teacher, sectional head and principal, without repeating for the same student inside a three-day window.
- Early-leave records capture a student's mid-day departure, time, reason and who collected them.

### 6.2 Staff Attendance **FR-SAT**

- Administrator or principal mark the whole staff board present/late/on-leave/absent for a chosen date in one bulk action.
- A date locks for everyone, principal included, once it is in the past or has already been submitted for today; reopening it is a principal-only action.
- Staff with no attendance row and no approved leave are automatically marked absent at the end of the school day by a scheduled job.
- A staff member's own profile shows their attendance history and leave taken, visible to those permitted to view it.

### 6.3 Leave Management **FR-LVE** **Extended**

- Staff request leave against a typed catalogue (casual, medical, duty, half-day, short leave, maternity) with balances tracked per staff member per academic year.
- **Short leave** is capped at a fixed number of half-days per calendar month rather than an annual allocation.
- **Maternity leave** is modelled as a three-phase chain — full pay, half pay, then no pay — each phase 84 days, with dates computed automatically from school-day rules rather than entered manually.
- A principal or administrator may file a leave request on behalf of another staff member.
- Approving a class teacher's leave requires a cover teacher to be nominated, or an explicit acknowledgement that none is needed — approval is blocked otherwise.
- At approval time, the approver may reclassify the leave type (e.g. from casual to medical) without asking the requester to resubmit.
- The principal can allocate or bulk-allocate annual leave entitlements to staff, by type and academic year.
- Approval atomically debits the balance, marks the staff member on leave for those dates, creates an audit entry, and notifies the requester; a cover assignment can also be removed later if plans change.

## 6.4 Student Records & Enrolment FR-STU

- Full student profile: identity, enrolment history (a promotion inserts a new enrolment row rather than overwriting the old one, preserving history), guardians, attendance calendar, marks, achievements, memberships and benefits, in one screen with sectioned tabs.
- Guardian records are shared across siblings — one guardian, many linked students — with address and administrative-division fields held on the guardian rather than duplicated per child.
- Search finds a student by partial name or admission number, scoped to what the searching user is permitted to see.
- Creating a new student record — singly or via spreadsheet import — is restricted to the principal and administrator specifically, even though editing an existing record is available more broadly.
- A student can be marked left (soft) or, if unassigned and with no attendance history, permanently deleted.
- Call logging records the purpose of a call placed to a guardian directly from the app.

## 6.5 Academic Structure FR-ADM

- Principal-managed catalogue of academic years, terms (with overlap validation), grades, subjects, classes, and which subjects each grade studies.
- Class-teacher assignment keeps the class record and the underlying role grant in sync automatically via a database trigger, removing a previous source of drift between the two.
- The "current" academic year is determined automatically from today's date rather than set manually.
- Grades, classes, subjects, years and terms can be deleted once nothing depends on them, guarded both by a UI check and the database's own foreign-key constraints.

## 6.6 Marks & Examinations FR-MRK Extended

- A mark sheet represents one class, one subject, one term. Entry defaults to a teacher's own subject assignment; submission locks the sheet, and only the principal can reopen it.
- Every student's score is shown beside the class average for the same subject and term, plus a term-over-term trend.
- **Exams** is a purpose-built browsing and analytics layer over the same mark-sheet data: a year → class → exam list, a guided "new exam" wizard, and a report-card-style view showing class average, top and bottom performers, and a three-band score distribution.

## 6.7 Student Promotion NEW

- A guided, two-step year-end rollover wizard: map every current class to a destination class in the next academic year (with a sensible default suggestion per grade), then review and confirm.
- Individual students can be excepted from the default mapping — for example, to repeat a grade.
- Destination classes that do not yet exist are created automatically; a new academic year can be created inline if needed.
- Promotion is idempotent — safe to re-run to pick up a straggling student without re-processing everyone else — and reports counts of promoted, already-enrolled and graduated students.

## 6.8 Staff Directory & Accounts FR-STF / FR-ACC

- Staff directory with presence indicator, profile, responsibilities (both ad-hoc duties and role-derived, e.g. "class teacher of 4B"), and a duty-roster view of the same data.
- Account provisioning creates the staff record, assigns roles with scope, and issues a login — the temporary password is emailed to the staff member and also shown once to the administrator as a fallback.
- Staff email and phone numbers are enforced unique at the database level, with friendly in-app error messages on conflict.
- Accounts can be deactivated; a deactivated account loses server-side access immediately, not only on next sign-in.

## 6.9 Announcements FR-ANN Extended

- Compose and publish to the whole school, one section, one class, or a chosen set of individuals, with optional scheduling for later publication.
- Read receipts let the author see who has read an announcement.
- An announcement can be edited within a short window after publishing, and deleted by its author on the same day or, without that time limit, by anyone holding school-wide publishing rights.

## 6.10 Events & School Diary FR-EVT / FR-DRY Extended

- A school calendar of events with edit, delete and "mark completed" (which automatically creates a matching diary entry), each event able to name several responsible staff members rather than one.
- Reminders fire at configurable lead times to every responsible staff member and to school leadership.
- The school diary records what actually happened, year-filterable, editable and deletable by anyone holding diary-management rights, not only the original author.
- Both the event calendar and the diary can be exported to PDF and shared through the device's native share sheet.

## 6.11 Inventory Management FR-INV

- Item catalogue with location, condition and code; stock quantity is always computed from an append-only transaction ledger (receipts, issues, returns, write-offs, adjustments), never stored and risking drift.
- Low-stock items surface first in the list and trigger a daily notification to the administrator and principal.

## 6.12 Analytics & Reporting FR-ANL

- School- and grade-level attendance percentage, a ranked at-risk student list, and marks performance summaries, backed by nightly-recomputed summary tables for fast loading.
- CSV export of attendance and marks summaries, generated server-side and delivered via a time-limited signed download link.

## 6.13 Notifications NEW screen

- A dedicated in-app notification centre lists every notification generated by the system's scheduled jobs and workflow events (reminders, risk alerts, leave decisions, event reminders, cover changes), with an unread badge reachable from every screen.
- Delivery also reaches the device's native notification tray via push, where a token has been registered.

## 6.14 Home Dashboard & Customisable Widgets NEW

- The home screen composes role-relevant blocks rather than being one fixed layout: a teacher's own unmarked-class prompt always leads for them; a principal sees the whole-school pulse and an actionable "needs attention" list first.
- A library of twenty widgets — enrolment snapshot, academic performance, birthdays, cover gaps, outstanding mark sheets, week's events, audit activity, announcements feed, and more — can be individually shown or hidden per staff member from a dedicated preferences screen, stored server-side against their account.

## 6.15 Audit Log FR-ADM-04

A generic, trigger-driven audit trail covers every table writable directly from the client with no bespoke logging of its own (staff records, role grants, calendar overrides, settings, achievements, memberships, benefits, inventory, events, diary, responsibilities, academic structure, guardians, students); attendance, marks, leave and announcements write their own audit entries as part of their transactional functions. The audit log is filterable by actor, entity and date, and viewable only with a dedicated permission.

## 6.16 Settings

Own profile display, notification preferences (including device push-token registration), sign-out, and — on sign-out — a warning if unsynced offline writes would be lost, followed by a full clear of local cache and offline storage.

## SECTION 7

# Non-Functional Requirements

| Category               | Requirement                                         | How it is met                                                                                                                                               |
|------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reliability            | Offline operations must never be lost or duplicated | Idempotent client-generated submission IDs; a SQLite queue that survives app restart; server treats a repeated ID as a no-op returning the original result. |
| Security               | Server-side enforcement of every access rule        | Row Level Security on every table, resolved through <code>has_permission()</code> ; client-side hiding of controls is presentation only, never protection.  |
| Security               | Deactivated accounts lose access immediately        | <code>current_staff_id()</code> excludes deactivated staff at the database layer; a live-session check forces prompt local sign-out.                        |
| Privacy                | Sensitive fields restricted beyond row scope        | Guardian contact numbers and student benefits sit behind dedicated permissions and views, independent of general record access.                             |
| Performance            | Dashboards load quickly even at whole-school scale  | Nightly-recomputed summary tables for attendance and marks, rather than aggregating raw records on every screen load.                                       |
| Usability              | The core daily task is fast                         | Attendance marking defaults to all-present and is designed to complete in under thirty seconds.                                                             |
| Accessibility          | WCAG AA contrast and tap-target size                | Status indicator colours audited and corrected to a minimum 4.5:1 contrast ratio; interactive elements meet a 44dp minimum tap target.                      |
| Auditability           | Every write of consequence is traceable             | Central audit log capturing actor, action, entity, and before/after state.                                                                                  |
| Configurability        | Thresholds and deadlines are not hard-coded         | Attendance due time, edit window, risk thresholds and leave entitlements are editable configuration, not literals in code.                                  |
| Localisation readiness | No user-facing string hard-coded                    | i18n infrastructure in place and used by the core navigation shell; full extraction across all feature screens remains in progress (see Section 15).        |
| Rate limiting          | Abuse of spammable actions is bounded               | Per-action rate limits applied to reminder dispatch and announcement creation, the two user-triggered actions realistically open to repeat-spamming.        |

## SECTION 8

# Automation & Scheduled Jobs

Every scheduled job begins by checking whether today is a school day and exits immediately if not — the system's guarantee that a holiday never generates a false absence record or an unwanted notification.

| Job                             | Schedule                     | Behaviour                                                                                                                                                               |
|---------------------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Remind unmarked classes         | At the attendance due time   | Notifies the class teacher (or active cover teacher) of any class with no attendance submitted yet today.                                                               |
| Escalate unmarked classes       | 30 minutes after deadline    | Sends a digest of still-unmarked classes to the principal and relevant sectional heads.                                                                                 |
| Lock attendance                 | Hourly                       | Locks any submission whose edit window has passed.                                                                                                                      |
| Detect absence risk             | Daily, 30 min after deadline | Flags students meeting the consecutive-absence or term-percentage threshold; notifies class teacher, sectional head and principal, at most once per student per 3 days. |
| Mark staff absent               | At end of school day         | Any staff member with no attendance row and no approved leave is marked absent.                                                                                         |
| Event reminders                 | Daily, 06:00                 | Notifies each event's responsible staff (and leadership) at its configured lead times.                                                                                  |
| Publish scheduled announcements | Every 5 minutes              | Publishes any announcement whose scheduled time has passed.                                                                                                             |
| Recompute summaries             | Nightly, 01:00               | Rebuilds attendance summary rollups for student, class, grade and school scope.                                                                                         |
| Low-stock check                 | Daily, 07:00                 | Notifies the administrator and principal of inventory at or below its minimum quantity.                                                                                 |
| Push dispatch                   | Every minute                 | Delivers queued notification rows to devices via the Expo push service.                                                                                                 |

## Offline Synchronisation Design

Three operations are offline-capable by design: marking attendance, recording an early leave, and entering marks. Every other action requires connectivity and says so plainly rather than pretending to succeed.

### 9.1 Rules

1. Every offline operation carries a device-generated unique submission ID. The server treats a repeat of the same ID as a no-op and returns the original result — idempotency is mandatory, not best-effort.
2. The queue drains oldest-first, one operation at a time, on app foreground, on network regain, and every 60 seconds while online.
3. Failed operations retry with exponential backoff for up to 24 hours, computed from the last attempt (not only the first), then surface to the user as a banner with a manual retry option.
4. A teacher's own class roster is cached locally and refreshed on every successful sync, so the marking screen opens instantly even offline.
5. Sync state is always visible — a synced tick, a pending badge with count, or an error banner — so the user never has to wonder whether their marking was saved.
6. If a submission arrives after the class was already submitted by someone else, the server returns a conflict with the existing submission rather than silently overwriting it.

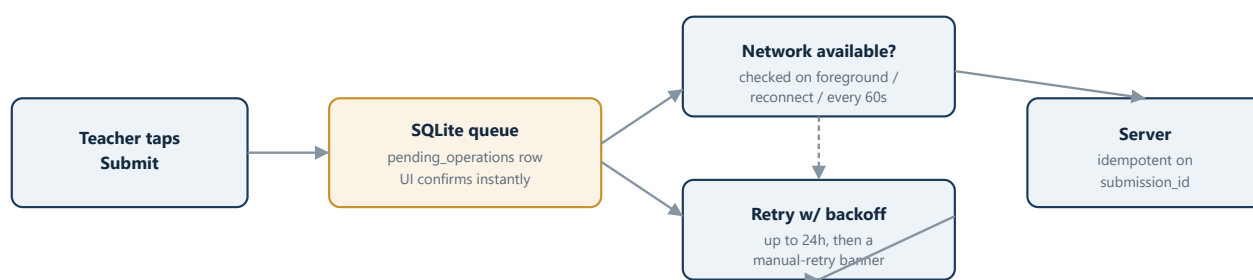


Figure 9.1 — Offline submission lifecycle: instant local confirmation, background drain, idempotent server write.

## Security Model

---

- **Row Level Security everywhere.** Every table has RLS enabled; a table with no explicit policy denies all access by default rather than being open.
- **Single scope resolver.** All policies call the same `has_permission()` function, so a permission rule is defined once and applied identically across every read and write path — PostgreSQL, Edge Functions, and any future client.
- **Privileged operations isolated to Edge Functions.** Anything requiring the Supabase service-role key — issuing a login, generating a signed export URL — runs server-side in a Deno function that independently re-verifies the caller's permission via their own JWT before ever touching a privileged client. The mobile app itself never holds a service-role credential.
- **Column-level protection** for guardian contact details and student benefit records, layered on top of row-level rules.
- **Full audit trail** of who changed what, when, and what the value was before and after.
- **Immediate deactivation.** A deactivated staff account is excluded from permission resolution at the database layer the moment it is deactivated, with a live-session check that signs the device out promptly rather than waiting for a token to expire.
- **Rate limiting** on the small number of actions a user could otherwise spam (manual reminders, announcement creation).

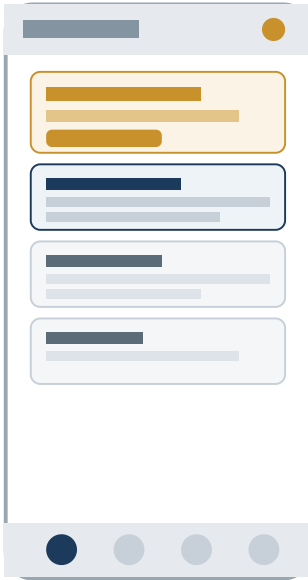
# User Interface Design — Wireframes

The following are low-fidelity structural wireframes only — they show screen layout and information hierarchy, not final visual design, colour or branding, which live in the implemented application itself.



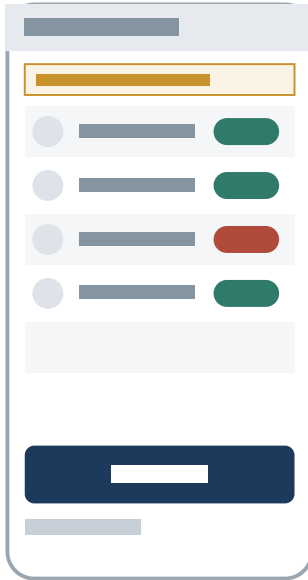
Sign In

Email + password, biometric unlock on return visits, forgot-password link.



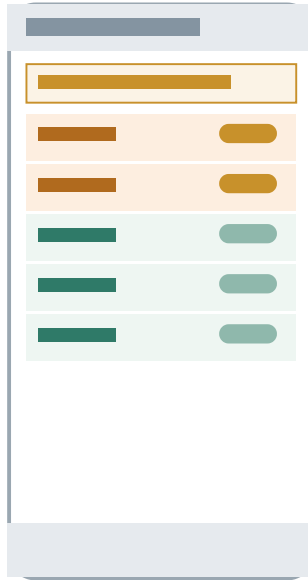
Home Dashboard

Role-composed blocks: own task first, school pulse, needs-attention list, then bottom-tab navigation.



Mark Attendance

Roster defaults to present; tap to toggle a status; sync indicator; single submit action.



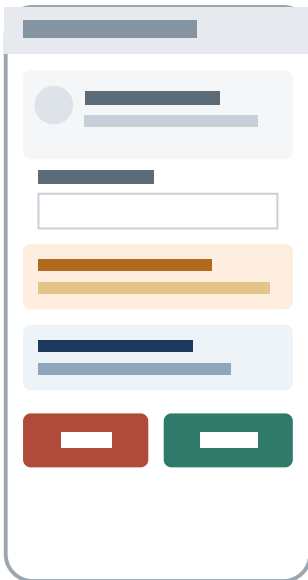
Attendance Board

Unmarked classes surface first (amber), marked classes follow (green); per-class remind action.



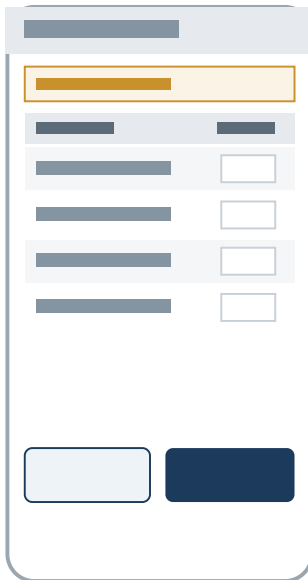
Student Profile

Tabbed sections — attendance, marks, activity, benefits — over one identity header; call/log actions.



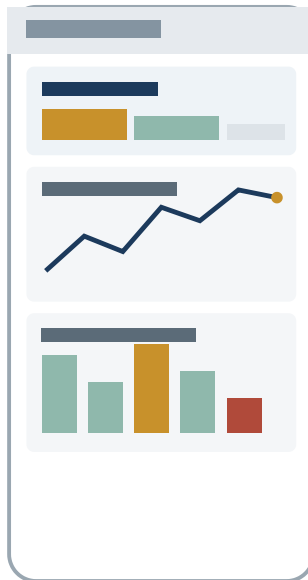
Leave Request Detail

Requester + balance context, coverage-gap warning, reject / approve actions with cover requirement.



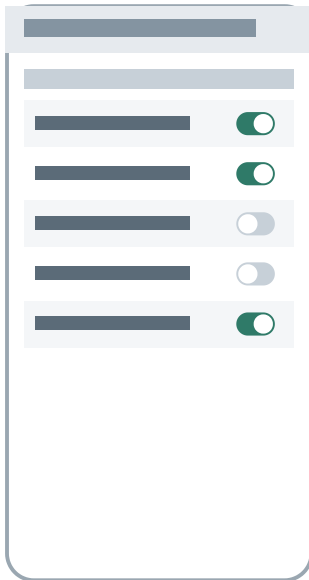
Marks Entry

Roster with per-student score field; save as draft or submit & lock.



Analytics

School/grade attendance %, trend line, at-risk breakdown, CSV export.



### Dashboard Widgets

Per-user toggle list controlling which Home cards are shown, grouped by category.

## SECTION 12

## Navigation Map

Every route in the application, the permission that gates it, and the screen it opens. Bottom-tab routes (Home, Student Search, Attendance Board, Announcements/Staff, More) are the primary entry points; every other route is reached from within them.

| Route                                             | Permission gate                | Purpose                                          |
|---------------------------------------------------|--------------------------------|--------------------------------------------------|
| SignIn / SetPassword                              | —                              | Authentication and first-login password set      |
| Home                                              | authenticated                  | Role-composed dashboard                          |
| DashboardWidgets <b>new</b>                       | authenticated                  | Choose which Home widgets are shown              |
| StudentSearch / ClassList / ClassDetail           | student / attendance view      | Find a student or browse by class                |
| StudentProfile / AddStudent / SetClass            | student.view / edit            | Full student record, creation, class assignment  |
| MarkAttendance / AttendanceSubmitted / EarlyLeave | attendance.mark                | Daily register and early departures              |
| MarkStaffAttendance                               | attendance.mark_staff          | Bulk staff attendance                            |
| AttendanceBoard                                   | attendance.view_board          | Whole-scope marking status, student/staff toggle |
| MarkEntry / MarksReview                           | marks.enter / review           | Score entry and cross-class review               |
| Exams / ExamMarks <b>new</b>                      | marks.enter / review           | Exam-centric browsing and report-card analytics  |
| StaffDirectory / StaffProfile                     | staff.view_directory           | Staff records and responsibilities               |
| MyLeave / LeaveRequests / LeaveRequestDetail      | leave.request / approve        | Leave request and approval workflow              |
| LeaveAllocation <b>new</b>                        | leave.approve                  | Set staff leave entitlements                     |
| AssignCover                                       | cover.assign                   | Nominate a cover teacher                         |
| Announcements / ComposeAnnouncement               | announcement.publish_*         | Feed and composer                                |
| Inventory / InventoryItem                         | inventory.view / manage        | Stock catalogue and movements                    |
| EventCalendar / EventDetail / Diary               | authenticated / event.manage   | School events and diary                          |
| AcademicCalendar / CalendarDayEditor              | calendar.manage                | Years, terms, working days, day overrides        |
| AcademicStructure                                 | calendar.manage                | Grades, subjects, classes, teacher assignment    |
| PromoteStudents <b>new</b>                        | student.edit (admin/principal) | Year-end class promotion wizard                  |
| Analytics                                         | analytics.view                 | School/grade dashboards and exports              |
| UserAccounts / AuditLog                           | account.manage / audit.view    | Account provisioning and audit trail             |
| Notifications <b>new</b>                          | authenticated                  | In-app notification centre                       |
| Settings                                          | authenticated                  | Profile, notification preferences, sign out      |

## SECTION 13

# Quality Assurance & Testing

| Layer                 | Approach                                                                                                                                                                                                                                                                      |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Static analysis       | TypeScript strict mode; ESLint; both run clean across the client codebase.                                                                                                                                                                                                    |
| Unit / logic tests    | Vitest, covering pure logic such as offline retry/backoff calculation, kept deliberately separate from native modules that cannot run under a test runner.                                                                                                                    |
| Database policy tests | pgTAP test suite proving Row Level Security actually isolates data — e.g. a class teacher's query for another class returns zero rows regardless of what the client requests — plus targeted tests for locking, reopen rules, and scheduled-job no-op behaviour on a holiday. |
| Build verification    | Full Expo export bundles the application with zero bundler errors, used as the closest available proxy for "the app boots" in a headless build environment.                                                                                                                   |
| Accessibility audit   | Every status-indicator colour pair measured against WCAG AA 4.5:1 contrast and corrected where it failed; tap targets verified at a 44dp minimum.                                                                                                                             |
| Data integrity checks | Every table's presence of Row Level Security cross-checked against its creation migration, table by table.                                                                                                                                                                    |

## Project Status & Build Progress

The system was built in a defined, sequential order — each stage's test had to pass before the next began — so that offline attendance marking, the highest-priority flow, was working before any secondary module was started.

### 14.1 Build Stages

| #  | Stage                                           | Status  | Note                                                                                                     |
|----|-------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------|
| 1  | Project scaffold, CI, migration pipeline        | Done    | Full navigation shell, theming, CI workflow in place.                                                    |
| 2  | Core tables, roles, permissions, seed data      | Done    | Permission resolver verified against hand-written cases.                                                 |
| 3  | Authentication & session handling               | Done    | Email sign-in, first-login password set, OTP reset, biometric unlock.                                    |
| 4  | Row Level Security across all tables            | Ongoing | Every table has policies; extended alongside each new table added since.                                 |
| 5  | Spreadsheet import                              | Done    | Students and staff, per-row reconciliation, no partial commits.                                          |
| 6  | Academic calendar & school-day logic            | Done    | Terms, working days, day-type overrides.                                                                 |
| 7  | Student/staff directories & search              | Done    |                                                                                                          |
| 8  | Offline engine                                  | Done    | SQLite queue, idempotent sync, backoff.                                                                  |
| 9  | Attendance marking & submission                 | Done    | Core flow — the system's highest priority.                                                               |
| 10 | Attendance board & reminders                    | Done    |                                                                                                          |
| 11 | Scheduled jobs (reminders, locking, risk)       | Done    | Holiday no-op behaviour verified.                                                                        |
| 12 | Early leave & staff attendance                  | Done    | Including day-lock/reopen rules.                                                                         |
| 13 | Leave: requests, balances, approval, cover      | Done    | Extended with allocation, maternity phases, short leave.                                                 |
| 14 | Marks: entry, lock, reopen, averages            | Done    | Extended with the Exams analytics layer.                                                                 |
| 15 | Achievements, memberships, benefits             | Done    |                                                                                                          |
| 16 | Announcements: compose, audience, receipts      | Done    | Extended with edit window and delete rules.                                                              |
| 17 | Responsibilities & duty roster                  | Done    |                                                                                                          |
| 18 | Inventory                                       | Done    | Barcode scanning not yet built.                                                                          |
| 19 | Events & school diary                           | Done    | Extended with multi-responsible staff, edit/delete/complete, PDF export.                                 |
| 20 | Analytics, summaries, exports                   | Done    | Period selector and subject-comparison view still pending.                                               |
| 21 | User accounts, roles, audit log                 | Done    | Login provisioning via email, including resend/reset.                                                    |
| 22 | Settings, notification preferences, i18n        | Ongoing | i18n infrastructure exists; full string extraction incomplete.                                           |
| 23 | Hardening: rate limiting, accessibility, backup | Ongoing | Rate limiting and accessibility audit complete; backup/restore untestable without a live hosted project. |

— Student promotion, dashboard widgets, exams UI

added  
beyond  
original plan

Built in response to real operational need once the core system was in use.

## 14.2 Scale of the Codebase

### Backend

- 102 versioned database migrations
- ~48 tables across 10 domains
- 30 permission keys, 6 roles
- 10 scheduled jobs
- 4 Edge Functions
- pgTAP policy test suite

### Mobile client

- ~35 feature modules
- ~40 navigable screens
- 20 configurable dashboard widgets
- Offline-capable core flows with idempotent sync
- TypeScript strict mode, zero lint warnings

## Known Limitations & Future Work

---

### 15.1 Currently Deferred

- **Localisation:** the i18n infrastructure is in place and used by the authentication and navigation shell, but most feature screens still hold English text inline pending full extraction.
- **Photo upload:** student photo and consent fields exist in the data model but no image-picker/upload path has been built yet, so photo consent is stored but not yet enforced anywhere.
- **Barcode scanning** for inventory remains manual entry/search by code rather than a camera scan.
- **Unified student timeline:** attendance, marks, activity and calls are all visible on a student's profile, but not yet interleaved into a single chronological feed.
- **Analytics depth:** no period selector or subject-by-class comparison view yet, though the underlying summary data already supports one.
- **Retention/purge policy** for students marked as left has not been implemented.
- **Sign-in identifier:** authentication is by email rather than staff number, a larger architectural change not yet attempted.

### 15.2 Explicitly Out of Scope for This Phase

Fee collection, timetabling, examination paper management, certificate generation, library circulation, transport management, biometric hardware integration, external authority reporting, and a web console are all deliberately excluded from this phase, to keep the system focused on the daily operational core.

### 15.3 Phase 2 — Parent Application

A parent-facing companion application is recognised as the natural next phase. The guardian and student data model was designed from the outset to support it without structural change — guardians are already first-class records linked to one or more students, ready to be given their own authenticated, appropriately-scoped view once that phase begins.

## Conclusion

---

The School Administration Management System replaces a school's paper-based attendance, records and communication workflow with a single mobile application that works reliably in real classroom conditions, including with no network connection. Every access rule is enforced at the database layer rather than trusted to the client, every write of consequence is audited, and the system's permission model scales from a single class teacher to whole-school oversight without code changes as the school's structure evolves.

What began as a specification centred on the single highest-priority flow — marking a class register in under thirty seconds — has grown, module by module and in the order that flow's own risk demanded, into a complete operational platform: attendance for both students and staff, leave management with real allocation and maternity-leave rules, marks and exam analytics, year-end promotion, inventory, events and diary, announcements, a customisable dashboard, and a full audit trail. The core daily workflow has been in place and tested since early in the build; the surrounding modules have been added, deliberately, once the foundation they depend on was solid.

Remaining work is well understood and scoped rather than open-ended: completing localisation string extraction, adding photo upload, deepening analytics, and — as a distinct future phase — extending the same data model to a parent-facing application. None of it blocks the system's use for its primary purpose today.

## Permission Key Catalogue

| Area              | Keys                                                                                                                                                                   |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Attendance        | attendance.mark , attendance.view_board , attendance.remind ,<br>attendance.amend_locked , attendance.early_leave , attendance.mark_staff ,<br>attendance.reopen_staff |
| Students          | student.view_basic , student.view_full , student.view_benefits , student.edit ,<br>student.view_guardian_contact                                                       |
| Marks             | marks.enter , marks.review , marks.reopen                                                                                                                              |
| Staff             | staff.view_directory , staff.view_full , staff.manage ,<br>staff.assign_responsibility                                                                                 |
| Leave & cover     | leave.request , leave.approve , leave.view_balances , cover.assign                                                                                                     |
| Announcements     | announcement.publish_all , announcement.publish_section                                                                                                                |
| Inventory         | inventory.view , inventory.manage                                                                                                                                      |
| Events & calendar | event.manage , diary.manage , calendar.manage                                                                                                                          |
| Reporting & admin | analytics.view , report.export , account.manage , audit.view                                                                                                           |

Permissions are seed data, not code — adding one is a migration and a seed row. Role-to-permission mapping is summarised in Section 5.2.

## Edge Functions

---

| Function           | Purpose                                                                                                                                                                                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| submit-attendance  | Idempotent student attendance submission — the offline queue's stable HTTP target; returns the absentee list with consecutive-absence counts.                                                                                         |
| import-records     | Bulk CSV import for students or staff; parses, validates every row, and returns a per-row accepted/rejected reconciliation report with no partial commit.                                                                             |
| export-report      | Generates an attendance-summary or marks-summary CSV and returns a one-hour signed download link, running under the caller's own permissions rather than a privileged bypass.                                                         |
| create-staff-login | Provisions or reissues a staff member's login, re-checking account-management permission before using the privileged service-role client; emails the temporary password and also surfaces it once to the administrator as a fallback. |

Straightforward reads and writes are served directly by Supabase's auto-generated API under Row Level Security; an Edge Function is used only where a transaction spans several tables or requires privileged, service-role work.